

SpaceGuard

Monitoramento Ambiental com Dados Espaciais e Sensores IoT

Integrantes do Grupo:

Nome completo	RM
Vitor Henrique Barreiro Marinho	rm550327
Pedro Fernandes Klein	rm99133
Luca de Alencar Salazar Primo	rm550443

1. Introdução

A economia espacial atravessa um momento de aceleração sem precedentes. Agências como NASA e ESA, aliadas a empresas privadas, expandem continuamente a rede de satélites de observação da Terra, gerando volumes massivos de dados sobre clima, cobertura vegetal, temperatura superficial e dinâmica oceânica. Paralelamente, a proliferação de dispositivos IoT (Internet of Things) permite que sensores físicos distribuídos em campo complementam esses dados com medições precisas e em tempo real.

A convergência dessas duas fontes — dados de satélite e leituras de sensores IoT — abre caminho para sistemas de monitoramento ambiental de nova geração: capazes de detectar riscos antes que se tornem desastres, comunicar alertas instantaneamente às autoridades e população, e manter histórico detalhado para análises preditivas.

O Brasil, detentor da maior biodiversidade do planeta e frequentemente afetado por incêndios, enchentes e secas extremas, é um dos países que mais se beneficiaria de uma infraestrutura robusta de monitoramento ambiental. Regiões como a Amazônia, o Pantanal e o Cerrado concentram ecossistemas de valor global, mas ainda carecem de sistemas integrados de vigilância e resposta rápida.

O **SpaceGuard** nasce nesse contexto como uma solução tecnológica que integra monitoramento por sensores IoT, alertas ambientais por região e uma interface mobile intuitiva, simulando a cadeia completa de um sistema de prevenção de desastres baseado em dados.

2. Problema

O Brasil registra, ano após ano, prejuízos humanos e econômicos de grande magnitude decorrentes de desastres ambientais que, em muitos casos, poderiam ter sido mitigados com detecção e resposta mais rápidas. Três problemas centrais motivam o desenvolvimento do SpaceGuard:

2.1 Detecção tardia de desastres ambientais

Incêndios florestais, enchentes e eventos climáticos extremos frequentemente são identificados apenas quando já atingiram proporções de difícil controle. A ausência de sensores distribuídos em campo e de sistemas de alerta precoce faz com que as primeiras notificações cheguem horas ou dias após o início do evento, reduzindo drasticamente a janela de resposta das equipes de proteção civil.

2.2 Falta de monitoramento integrado em regiões vulneráveis

Regiões de alta vulnerabilidade ambiental — como áreas de transição entre biomas, margens de rios sujeitas a cheias sazonais ou zonas de desmatamento ativo — carecem de infraestrutura de monitoramento contínuo. Sem dados em tempo real, órgãos ambientais e defesa civil operam de forma reativa, respondendo a situações já consolidadas em vez de preveni-las.

2.3 Dificuldade de resposta rápida e coordenação

Mesmo quando os dados existem, sua fragmentação em diferentes sistemas e formatos dificulta a tomada de decisão rápida. A ausência de uma plataforma unificada que agregue leituras de sensores, classifique o nível de risco por região e comunique alertas de forma padronizada representa um gargalo operacional crítico.

3. Solução Proposta

O **SpaceGuard** é uma plataforma de monitoramento ambiental full stack que integra um aplicativo mobile, uma API REST e um banco de dados relacional para simular um sistema completo de vigilância ambiental baseado em sensores IoT e dados regionais.

A solução é composta pelos seguintes componentes:

Aplicativo Mobile (SwiftUI — iOS)

Interface intuitiva desenvolvida para iPhone, permitindo que operadores e gestores visualizem alertas ambientais em tempo real, acompanhe o status dos sensores por região e acessem o histórico de leituras. A arquitetura MVVM garante separação clara entre lógica de negócio e interface, facilitando manutenção e evolução.

API REST (Java Spring Boot)

Backend robusto que expõe endpoints RESTful para todas as operações do sistema: autenticação de usuários, gerenciamento de regiões, sensores e leituras, e criação e consulta de alertas ambientais. A API segue os princípios REST e é documentada automaticamente via Swagger/OpenAPI.

Banco de Dados (PostgreSQL)

Armazenamento relacional de todas as entidades do sistema, com relacionamentos bem definidos, constraints de integridade e índices otimizados para as consultas mais frequentes. O esquema reflete fielmente a estrutura de um sistema de monitoramento ambiental real.

Sensores IoT Simulados

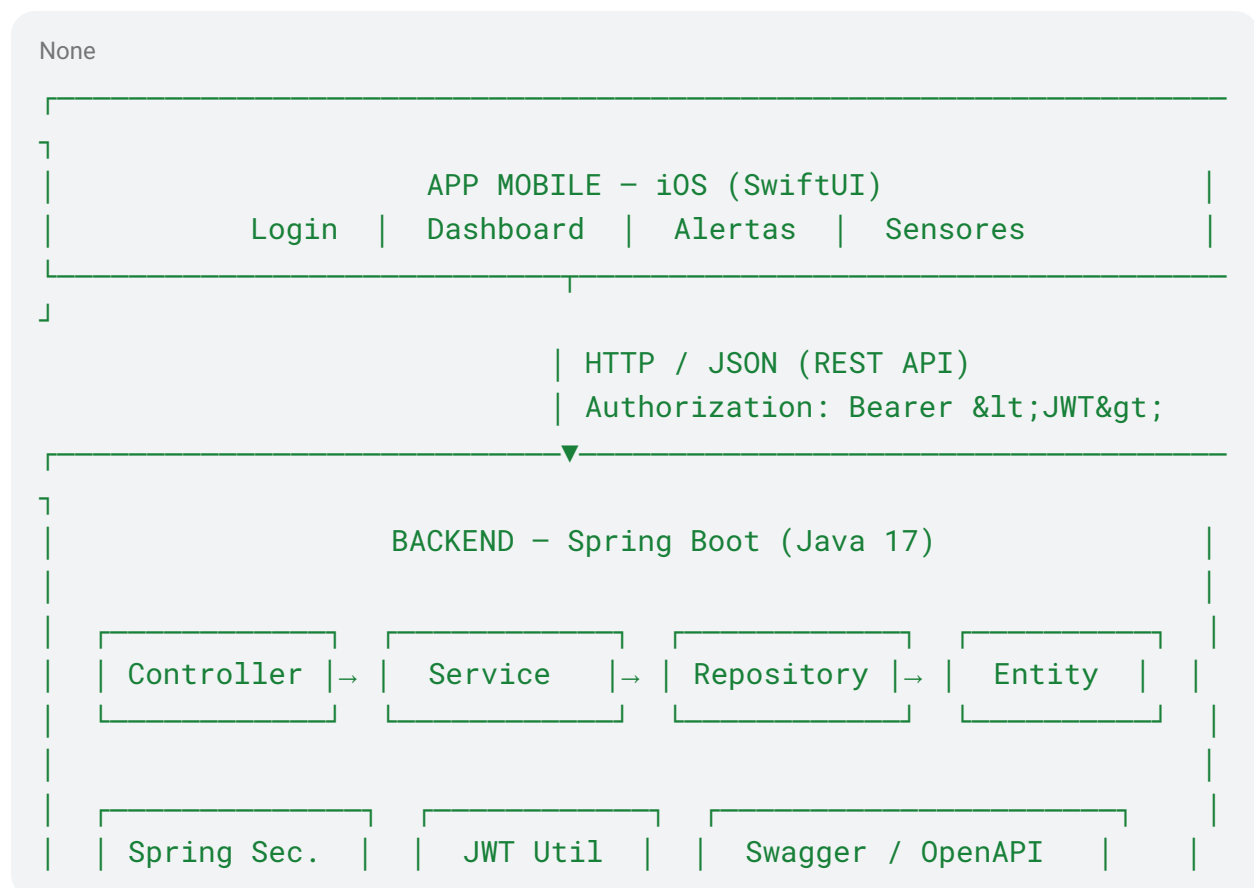
Cada sensor é cadastrado no sistema com tipo (temperatura, fumaça, nível de água, umidade), status operacional e vínculo com uma região. As leituras são registradas via API, simulando o envio periódico de telemetria que um sensor real faria. Leituras críticas disparam o contexto para geração de alertas.

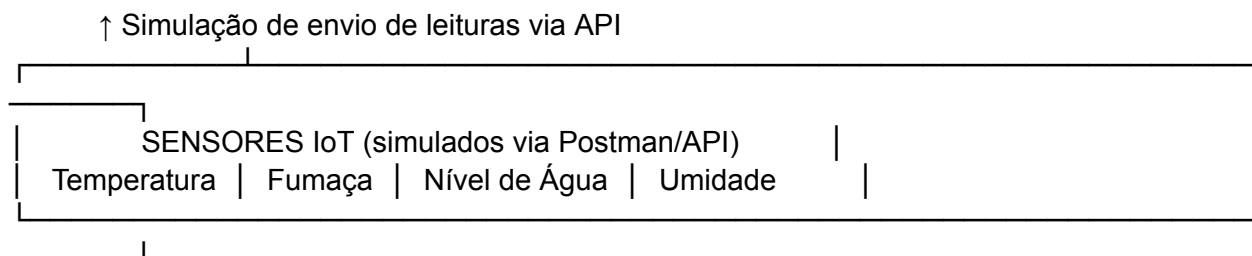
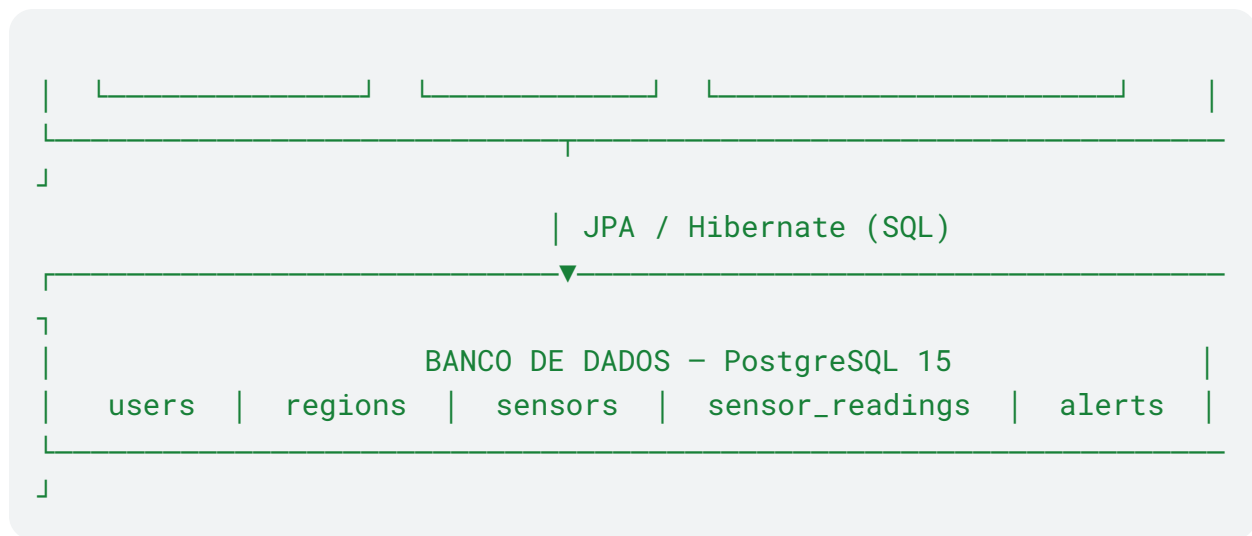
Alertas Ambientais por Região

O sistema classifica cada alerta com tipo (incêndio, enchente, fumaça, evento crítico) e nível de risco (baixo, médio, alto, crítico), associando-o a uma região geográfica monitorada. O aplicativo mobile exibe esses alertas em tempo real, priorizando os de maior severidade.

4. Arquitetura Geral

O SpaceGuard adota uma arquitetura em camadas distribuídas, separando claramente as responsabilidades de cada componente do sistema.





Fluxo principal:

1. O sensor IoT (ou sua simulação) envia uma leitura via `POST /sensors/{id}/readings`
2. O backend valida, autentica e persiste a leitura no PostgreSQL
3. Com base nas leituras, operadores criam alertas via `POST /alerts`
4. O app mobile consulta `GET /alerts` e exibe ao usuário final
5. Toda comunicação é protegida por token JWT gerado no login

5. Banco de Dados

O banco de dados do SpaceGuard foi modelado para refletir as entidades centrais de um sistema de monitoramento ambiental, com relacionamentos bem definidos e restrições de integridade que garantem a consistência dos dados.

5.1 Entidades

User (Usuário)

Representa os usuários do sistema — operadores e administradores. Armazena e-mail único e senha criptografada com BCrypt. É a entidade responsável pela autenticação via JWT.

Coluna	Tipo	Restrição
id	BIGSERIAL	Chave primária, auto-incremento
email	VARCHAR(255)	Obrigatório, único
password_hash	VARCHAR(255)	Obrigatório (BCrypt)
created_at	TIMESTAMP	Preenchido automaticamente

Region (Região)

Representa uma área geográfica monitorada. Cada região possui um nível de risco que sintetiza sua situação ambiental atual. É a entidade central do sistema, pois tanto os sensores quanto os alertas são vinculados a uma região.

Coluna	Tipo	Restrição
id	BIGSERIAL	Chave primária
name	VARCHAR(255)	Obrigatório
country	VARCHAR(255)	Obrigatório
risk_level	VARCHAR(50)	LOW \ MEDIUM \ HIGH \ CRITICAL

Sensor

Representa um dispositivo IoT instalado em uma região. Cada sensor tem um tipo (o que ele mede) e um status operacional. Um sensor pode registrar múltiplas leituras ao longo do tempo.

Coluna	Tipo	Restrição
id	BIGSERIAL	Chave primária
name	VARCHAR(255)	Obrigatório
type	VARCHAR(50)	TEMPERATURE \ SMOKE \ WATER \ HUMIDITY \ MULTI
status	VARCHAR(50)	ACTIVE \ INACTIVE \ MAINTENANCE
region_id	BIGINT	Chave estrangeira → regions(id)

SensorReading (Leitura de Sensor)

Registra uma medição pontual feita por um sensor. Armazena temperatura, nível de fumaça e nível de água (campos opcionais conforme o tipo do sensor). O campo `recorded\_at` é preenchido automaticamente no momento do registro.

Coluna	Tipo	Restrição
id	BIGSERIAL	Chave primária
temperature	DOUBLE PRECISION	Opcional
smoke_level	DOUBLE PRECISION	Opcional
water_level	DOUBLE PRECISION	Opcional
recorded_at	TIMESTAMP	Preenchido automaticamente
sensor_id	BIGINT	Chave estrangeira → sensors(id)

EnvironmentalAlert (Alerta Ambiental)

Registra um evento de risco detectado em uma região. Cada alerta possui tipo, nível de severidade e descrição detalhada. É a entidade consumida diretamente pelo app mobile para exibir notificações e o dashboard de riscos.

Coluna	Tipo	Restrição
id	BIGSERIAL	Chave primária
title	VARCHAR(255)	Obrigatório
description	TEXT	Opcional
risk_level	VARCHAR(50)	LOW \ MEDIUM \ HIGH \ CRITICAL
alert_type	VARCHAR(50)	FIRE \ FLOOD \ SMOKE \ CRITICAL_EVENT
created_at	TIMESTAMP	Preenchido automaticamente
region_id	BIGINT	Chave estrangeira → regions(id)

5.2 Relacionamentos

None

```
regions 1 — N sensors
regions 1 — N environmental_alerts
sensors 1 — N sensor_readings
```

- Uma **região** possui vários **sensores** instalados
- Uma **região** possui vários **alertas** ambientais associados
- Um **sensor** registra várias **leituras** ao longo do tempo

5.3 Diagrama ER

[INSERIR AQUI A IMAGEM DO DIAGRAMA ER — gerado em dbdiagram.io]

>

> O modelo DBML para geração do diagrama está disponível no arquivo

> `docs/spaceguard\_dbdiagram.dbml`. Acesse dbdiagram.io, cole o conteúdo

> do arquivo e exporte a imagem para inserir neste documento.

5.4 Arquivos SQL

Os scripts completos estão disponíveis na pasta `docs/` do projeto:

| Arquivo | Conteúdo |

|---|---|

| `spaceguard\_schema.sql` | CREATE TABLE das 5 entidades com PKs, FKs e constraints |

| `spaceguard\_inserts.sql` | Dados de exemplo: 2 usuários, 3 regiões, 5 sensores, 8 leituras, 5 alertas |

| `spaceguard\_queries.sql` | 5 consultas SQL de uso operacional |

6. API REST

6.1 Arquitetura em Camadas

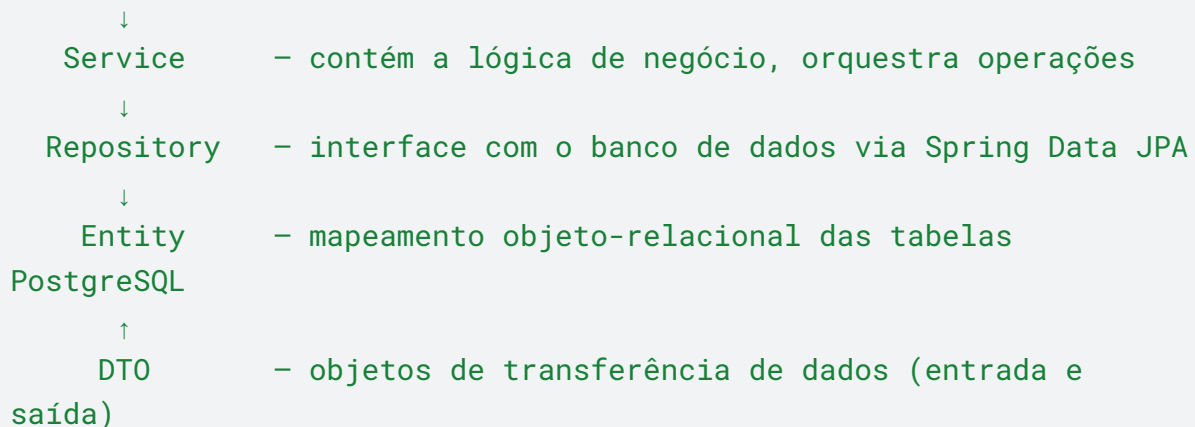
O backend segue o padrão de arquitetura em camadas amplamente adotado em aplicações Spring Boot, garantindo separação de responsabilidades e facilidade de manutenção:

None

Requisição HTTP

↓

Controller - recebe e valida a requisição, delega ao Service



- **Controller:** classes anotadas com `@RestController`, responsáveis por mapear rotas HTTP e formatar respostas JSON. Não contêm lógica de negócio.
- **Service:** classes `@Service` com as regras de negócio. Convertem entidades em DTOs e vice-versa, garantindo que o banco nunca seja exposto diretamente.
- **Repository:** interfaces que estendem `JpaRepository`, fornecendo operações CRUD e consultas personalizadas sem escrever SQL manual.
- **Entity:** classes mapeadas com `@Entity` que representam as tabelas do banco. Utilizam Lombok para eliminar código repetitivo.
- **DTO (Data Transfer Object):** objetos simples que definem exatamente o formato do JSON recebido e retornado pela API, isolando o modelo de dados interno.

6.2 Endpoints

Autenticação

Método	Endpoint	Descrição	Autenticação
POST	<code>/auth/register</code>	Cadastrar novo usuário	Pública
POST	<code>/auth/login</code>	Login e geração de token JWT	Pública

Regiões

Método	Endpoint	Descrição	Autenticação
GET	<code>/regions</code>	Listar todas as regiões	JWT
GET	<code>/regions/{id}</code>	Buscar região por ID	JWT
POST	<code>/regions</code>	Criar nova região	JWT

| PUT | `/regions/{id}` | Atualizar região | JWT |
| DELETE | `/regions/{id}` | Remover região | JWT |

Sensores

Método	Endpoint	Descrição	Autenticação
GET	`/sensors`	Listar todos os sensores	JWT
GET	`/sensors/{id}`	Buscar sensor por ID	JWT
POST	`/sensors`	Cadastrar novo sensor	JWT
PUT	`/sensors/{id}`	Atualizar sensor	JWT
DELETE	`/sensors/{id}`	Remover sensor	JWT
GET	`/sensors/{id}/readings`	Listar leituras do sensor	JWT
POST	`/sensors/{id}/readings`	Registrar nova leitura	JWT

Alertas

Método	Endpoint	Descrição	Autenticação
GET	`/alerts`	Listar todos os alertas	JWT
GET	`/alerts/{id}`	Buscar alerta por ID	JWT
GET	`/alerts/region/{id}`	Alertas por região	JWT
POST	`/alerts`	Criar novo alerta	JWT
PUT	`/alerts/{id}`	Atualizar alerta	JWT
DELETE	`/alerts/{id}`	Remover alerta	JWT

6.3 Autenticação via JWT

O sistema utiliza **JSON Web Token (JWT)** para autenticação stateless, adequada para APIs REST e aplicações mobile. O fluxo de autenticação funciona da seguinte forma:

1. O usuário envia e-mail e senha para `POST /auth/login`
2. O backend valida as credenciais e, se corretas, gera um token JWT assinado
3. O token é retornado ao cliente com validade de 24 horas
4. Em todas as requisições subsequentes, o cliente envia o token no header:
`Authorization: Bearer <token>`
5. O filtro `JwtFilter` intercepta cada requisição, valida o token e autentica o usuário no contexto de segurança do Spring

6.4 Swagger / OpenAPI

A API é documentada automaticamente pelo **SpringDoc OpenAPI**. Com o servidor em execução, acesse:

```
...  
http://localhost:8080/swagger-ui.html  
...
```

O Swagger permite explorar e testar todos os endpoints diretamente no navegador, com suporte a autenticação Bearer Token integrado à interface.

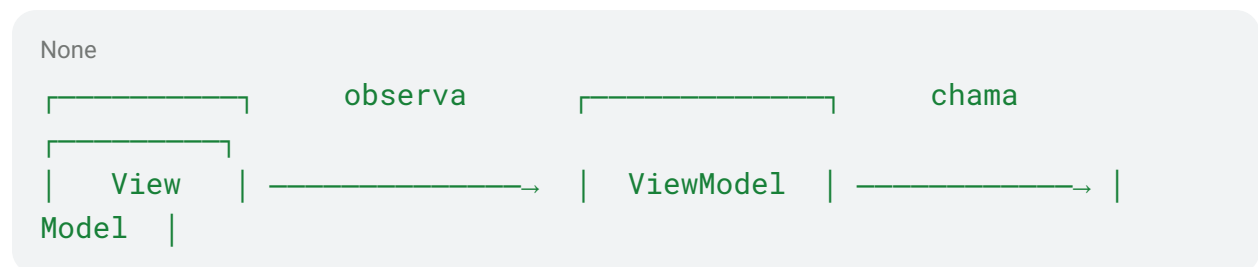
7. Front-end Mobile

7.1 Visão Geral

O front-end do SpaceGuard é um aplicativo iOS nativo desenvolvido em **SwiftUI**, a framework declarativa da Apple para construção de interfaces. O app foi projetado para exibir alertas ambientais em tempo real, detalhes por região e o painel de sensores IoT, com foco em usabilidade e clareza visual.

7.2 Arquitetura MVVM

O aplicativo segue o padrão arquitetural **MVVM (Model-View-ViewModel)**, amplamente adotado no desenvolvimento iOS com SwiftUI:



```

| (SwiftUI) | ← atualiza — | (@Observable | ← retorna — |
(Swift |
| | | / @State) | |
Struct) |
└──────────┘ └──────────┘
└──────────┘

```

- **Model:** estruturas Swift que representam os dados (Alert, Region, Sensor)
- **ViewModel:** classes com `@Observable` que contêm a lógica de apresentação, fazem chamadas à API e expõem os dados formatados para a View
- **View:** componentes SwiftUI declarativos, sem lógica de negócio, que reagem automaticamente às mudanças nos ViewModels

7.3 Telas do Aplicativo

Tela de Login

Permite ao usuário inserir e-mail e senha para autenticação. Ao fazer login, o token JWT é armazenado localmente e utilizado em todas as requisições subsequentes. Inclui tratamento de erros para credenciais inválidas.

> **[INSERIR AQUI PRINT DA TELA DE LOGIN]**

Dashboard — Mapa de Alertas

Tela principal do app, exibindo um painel com os alertas ambientais ativos, organizados por nível de risco (CRITICAL, HIGH, MEDIUM, LOW) e região. Cards coloridos indicam a severidade de cada evento, facilitando a triagem visual pelo operador.

> **[INSERIR AQUI PRINT DO DASHBOARD]**

Detalhe do Alerta

Exibe todas as informações de um alerta selecionado: título, descrição, tipo de evento, nível de risco, região afetada e data de criação. Permite ao operador ter contexto completo para tomada de decisão.

> **[INSERIR AQUI PRINT DO DETALHE DO ALERTA]**

Painel de Sensores IoT

Lista os sensores cadastrados por região, com seu status operacional e as

últimas leituras registradas (temperatura, fumaça, nível de água). Simula o painel de monitoramento de um sistema SCADA em versão mobile.

> **[INSERIR AQUI PRINT DO PAINEL DE SENSORES]**

7.4 Dados Mockados no Mobile

Durante o desenvolvimento do app mobile, os dados são fornecidos por objetos mockados diretamente no código Swift, sem dependência do backend. Essa abordagem permite desenvolver e testar a interface de forma independente, acelerando o ciclo de desenvolvimento. Ao integrar com o backend real, o ViewModel simplesmente substitui a fonte de dados mockada pela chamada à API REST, sem alteração nas Views.

8. Segurança

A segurança do SpaceGuard foi projetada em múltiplas camadas, cobrindo desde a autenticação até a proteção contra as vulnerabilidades mais comuns em aplicações web.

8.1 Autenticação com JWT

O sistema utiliza **JSON Web Tokens (JWT)** com algoritmo de assinatura **HS512** (HMAC com SHA-512). O token contém o e-mail do usuário como subject, data de emissão e data de expiração (24 horas). Cada requisição às rotas protegidas passa pelo `JwtFilter`, que valida a assinatura e a validade do token antes de permitir o acesso.

8.2 Criptografia de Senhas com BCrypt

As senhas dos usuários jamais são armazenadas em texto puro. O sistema utiliza **BCrypt** com fator de custo 10, um algoritmo de hashing adaptativo projetado especificamente para senhas. Mesmo que o banco de dados seja comprometido, as senhas permanecem protegidas contra ataques de força bruta.

8.3 Sessão Stateless

A configuração do Spring Security utiliza política de sessão **STATELESS**, o que significa que o servidor não mantém estado de sessão entre requisições. Cada chamada à API é completamente autônoma e autenticada individualmente pelo token JWT. Essa abordagem é ideal para APIs consumidas por aplicações mobile.

8.4 Validação de Entrada

Todas as entradas da API são validadas com **Bean Validation (Jakarta Validation)**. Campos obrigatórios, formatos de e-mail e tamanho mínimo de senha são verificados antes de qualquer processamento. Erros de validação retornam HTTP 400 com uma lista detalhada dos campos inválidos, facilitando o tratamento no lado do cliente.

8.5 Proteção contra SQL Injection

O sistema utiliza **Spring Data JPA com Hibernate**, que executa todas as consultas ao banco de dados através de **Prepared Statements** parametrizados. Essa abordagem elimina completamente o risco de SQL Injection, pois os valores fornecidos pelo usuários nunca são concatenados diretamente em strings SQL.

8.6 Rotas Públicas e Protegidas

Rota Acesso
--- ---
`POST /auth/login` Pública
`POST /auth/register` Pública
`GET /swagger-ui/**` Pública
`GET /v3/api-docs/**` Pública
Todas as demais Exige JWT válido

9. IoT — Sensores Ambientais

9.1 Visão Geral

O SpaceGuard simula uma rede de sensores IoT distribuídos por regiões geográficas monitoradas. Cada sensor é cadastrado no sistema com um tipo específico de medição e envia leituras periódicas que alimentam o painel de monitoramento e contextualizam a geração de alertas.

Em um ambiente de produção real, os sensores físicos enviaram seus dados via protocolos como **MQTT** ou **HTTP**, conectando-se diretamente ao backend. No SpaceGuard, esse envio é simulado através de chamadas à API REST, permitindo demonstrar o fluxo completo sem necessidade de hardware.

9.2 Tipos de Sensores

Tipo	O que mede	Unidade	Uso
TEMPERATURE	Temperatura do ar	°C	Detecção de calor extremo e risco de incêndio
SMOKE	Índice de fumaça	0.0 a 1.0	Identificação de focos de incêndio
WATER	Nível de água	metros	Monitoramento de enchentes e cheias
HUMIDITY	Umidade relativa	%	Avaliação de risco de incêndio e seca
MULTI	Múltiplos parâmetros	—	Sensor combinado (temperatura + fumaça + água)

9.3 Status Operacional

Cada sensor possui um dos três status possíveis:

- **ACTIVE**: operando normalmente, enviando leituras
- **INACTIVE**: desativado temporariamente, sem envio de dados
- **MAINTENANCE**: em manutenção, podendo enviar leituras inconsistentes

9.4 Leituras e Geração de Alertas

Uma leitura de sensor é registrada via `POST /sensors/{id}/readings` com os valores medidos naquele instante. O campo `recorded_at` é preenchido automaticamente pelo servidor, garantindo consistência temporal.

Exemplos de leituras que indicam situação crítica:

Parâmetro	Valor normal	Valor crítico	Alerta gerado
Temperatura	< 35°C	> 45°C	FIRE / CRITICAL_EVENT
Fumaça	< 0.3	> 0.8	SMOKE / FIRE
Nível d'água	< 1.5m	> 2.5m	FLOOD

O ciclo completo de monitoramento IoT no SpaceGuard funciona da seguinte forma:

None

Sensor registra leitura

↓

POST /sensors/{id}/readings

↓

Dados persistidos no PostgreSQL

↓

Operador analisa leituras críticas

↓

POST /alerts (alerta criado para a região)

↓

GET /alerts (app mobile exibe o alerta)

10. Testes

10.1 Estratégia de Testes

Os testes do SpaceGuard foram realizados de forma manual utilizando a ferramenta **Postman** e o utilitário de linha de comando **curl**, validando cada endpoint da API com diferentes cenários de entrada: dados válidos, dados inválidos, requisições sem autenticação e requisições com token expirado ou incorreto.

10.2 Resumo do Plano de Testes

ID	Funcionalidade	Cenário	HTTP Esperado	Resultado
TC-001	Cadastro	Novo usuário com dados válidos	200	✓ PASSOU
TC-002	Cadastro	E-mail já cadastrado	400	✓ PASSOU
TC-003	Login	Credenciais corretas	200	✓ PASSOU
TC-004	Login	Senha incorreta	400	✓ PASSOU
TC-005	Segurança	Acesso sem token JWT	403	✓ PASSOU
TC-006	Alertas	Listagem autenticada	200	✓ PASSOU
TC-007	Alertas	Criação de novo alerta	201	✓ PASSOU
TC-008	Sensores	Registro de leitura IoT	201	✓ PASSOU
TC-009	Alertas	Filtro por região	200	✓ PASSOU
TC-010	Validação	Campos obrigatórios faltando	400	✓ PASSOU

****Taxa de sucesso: 10/10 — 100%****

O documento completo com pré-condições, passos detalhados, entradas e saídas de cada caso de teste está disponível no arquivo `docs/plano_de_testes.md`.

10.3 Testes via Postman

Para reproduzir os testes, configure o Postman com as seguintes etapas:

1. Crie uma Collection chamada ****SpaceGuard API****
2. Adicione a variável de coleção `token`
3. No request de login, adicione no campo ****Tests****:

JavaScript

```
pm.collectionVariables.set("token", pm.response.json().token);
```

4. Em todos os requests protegidos, adicione o header:

...

```
Authorization: Bearer {{token}}
```

...

A documentação Swagger disponível em `http://localhost:8080/swagger-ui.html` também pode ser importada diretamente no Postman via URL `http://localhost:8080/v3/api-docs`.

10.4 Evidências de Execução

****Evidência 1 — Cadastro de usuário (TC-001): HTTP 200****

> ****[INSERIR AQUI PRINT DO POSTMAN — POST /auth/register retornando token]****

```
```json
POST /auth/register → 200 OK
{
 "token": "eyJhbGciOiJIUzUxMiJ9...",
 "email": "teste@spaceguard.com",
 "type": "Bearer"
}
```
```

****Evidência 2 — Listagem e criação de alertas (TC-006, TC-007): HTTP 200 / 201****

> ****[INSERIR AQUI PRINT DO POSTMAN — GET /alerts e POST /alerts]****

```
```json
POST /alerts → 201 Created
{
 "id": 6,
 "title": "Alerta de Fumaça — Novo Foco",
 "riskLevel": "HIGH",
 "alertType": "SMOKE",
 "regionId": 1,
 "createdAt": "2026-06-01T21:19:37.536667"
}
```
```

****Evidência 3 — Segurança e validação (TC-005, TC-010): HTTP 403 / 400****

> ****[INSERIR AQUI PRINT DO POSTMAN — GET /alerts sem token e POST /alerts incompleto]****

```
```json
GET /alerts sem Authorization → 403 Forbidden

POST /alerts com body incompleto → 400 Bad Request
{
 "title": "Title is required",
 "riskLevel": "Risk level is required",
}
```

```
"alertType": "Alert type is required",
"regionId": "Region ID is required"
}
...
```

## 11. Conexão com ODS da ONU

O SpaceGuard está diretamente alinhado com três Objetivos de Desenvolvimento Sustentável da Agenda 2030 da ONU:

### ### ODS 9 — Indústria, Inovação e Infraestrutura

> **"Construir infraestruturas resilientes, promover a industrialização inclusiva e sustentável e fomentar a inovação."**

O SpaceGuard exemplifica o uso de infraestrutura tecnológica de ponta — computação em nuvem, IoT e aplicações mobile — a serviço do monitoramento ambiental. A integração de dados de sensores com uma API REST escalável demonstra como a inovação tecnológica pode ser aplicada a problemas ambientais concretos, criando infraestrutura digital resiliente para gestão de riscos.

### ### ODS 11 — Cidades e Comunidades Sustentáveis

> **"Tornar as cidades e os assentamentos humanos inclusivos, seguros, resilientes e sustentáveis."**

Enchentes, incêndios e eventos climáticos extremos ameaçam diretamente a segurança de comunidades urbanas e rurais. O SpaceGuard contribui para ODS 11 ao fornecer ferramentas de alerta precoce que permitem evacuações planejadas, alocação eficiente de recursos de resposta e tomada de decisão baseada em dados reais — tornando comunidades mais resilientes a desastres ambientais.

### ### ODS 13 — Ação Contra a Mudança Global do Clima

> **"Tomar medidas urgentes para combater a mudança climática e seus impactos."**

O monitoramento contínuo de temperatura, fumaça e nível de água em regiões vulneráveis é uma resposta direta aos impactos da mudança climática, que intensifica a frequência e severidade de eventos como secas, incêndios florestais e enchentes. O SpaceGuard oferece a infraestrutura de dados necessária para que gestores públicos e equipes de proteção civil ajam com mais rapidez e precisão diante desses eventos.

## 12. Conclusão

O SpaceGuard demonstra, em escala de projeto acadêmico, como tecnologias amplamente disponíveis — Spring Boot, PostgreSQL, SwiftUI e JWT — podem ser combinadas para construir uma solução robusta e coerente de monitoramento ambiental.

O projeto cobre o ciclo completo de uma aplicação full stack moderna: desde a modelagem relacional do banco de dados e a implementação de uma API REST segura, passando pelo desenvolvimento mobile com arquitetura MVVM, até a simulação de dispositivos IoT e a documentação de testes com evidências reais de execução.

Do ponto de vista técnico, os principais aprendizados incluem a integração entre camadas (mobile, backend, banco de dados), a implementação de autenticação stateless com JWT, o mapeamento objeto-relacional com JPA e a documentação automática de APIs com Swagger.

Do ponto de vista de impacto, o SpaceGuard evidencia como a tecnologia pode contribuir diretamente para a prevenção de desastres ambientais e para a proteção de vidas e ecossistemas — objetivos que transcendem o contexto acadêmico e se alinham com os desafios mais urgentes da atualidade.

Como evolução futura, o projeto poderia incorporar:

- **Integração com dados reais de satélite** via APIs da NASA (FIRMS) ou ESA (Copernicus)
- **Notificações push** no app iOS para alertas críticos em tempo real
- **Machine Learning** para predição de risco com base no histórico de leituras
- **Protocolo MQTT** para comunicação real com sensores IoT físicos
- **Mapa interativo** com visualização geoespacial dos alertas por região

## 13. Referências

### **\*\*Agências e Organizações:\*\***

- NASA — National Aeronautics and Space Administration. \*FIRMS: Fire Information for Resource Management System.\* Disponível em: [firms.modaps.eosdis.nasa.gov](https://firms.modaps.eosdis.nasa.gov)
- ESA — European Space Agency. \*Copernicus: Europe's Eyes on Earth.\* Disponível em: [www.esa.int/Applications/Observing\\_the\\_Earth/Copernicus](https://www.esa.int/Applications/Observing_the_Earth/Copernicus)
- International Charter Space and Major Disasters. \*Activations and Emergency Response.\* Disponível em: [disasterscharter.org](https://disasterscharter.org)
- ONU — Nações Unidas. \*Objetivos de Desenvolvimento Sustentável — Agenda 2030.\* Disponível em: [brasil.un.org/pt-br/sdgs](https://brasil.un.org/pt-br/sdgs)

### **\*\*Documentação Técnica:\*\***

- Spring Boot Documentation. \*Spring Framework Reference.\* Disponível em: [docs.spring.io/spring-boot/docs/current/reference/html](https://docs.spring.io/spring-boot/docs/current/reference/html)
- PostgreSQL Global Development Group. \*PostgreSQL 15 Documentation.\* Disponível em: [www.postgresql.org/docs/15](https://www.postgresql.org/docs/15)
- Apple Inc. \*SwiftUI Documentation.\* Disponível em: [developer.apple.com/documentation/swiftui](https://developer.apple.com/documentation/swiftui)
- SpringDoc OpenAPI. \*springdoc-openapi v2 Documentation.\* Disponível em: [springdoc.org](https://springdoc.org)
- JWT.io. \*JSON Web Token Introduction.\* Disponível em: [jwt.io/introduction](https://jwt.io/introduction)
- OWASP Foundation. \*OWASP Top Ten Security Risks.\* Disponível em: [owasp.org/www-project-top-ten](https://owasp.org/www-project-top-ten)

\*Documento gerado para fins acadêmicos — SpaceGuard Environmental Monitoring System\*